



Python Programming Fundamentals

Scope of Variables

Programming Fundamentals Team

SETU

2026-06-18



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



1. What is Scope?

Scope refers to the region of a program where a variable is accessible.



1. What is Scope?

Scope refers to the region of a program where a variable is accessible.

Python uses **LEGB** scope rules:

- **L** — Local: variables defined inside the current function
- **E** — Enclosing: variables in any enclosing functions
- **G** — Global: variables defined at module level
- **B** — Built-in: Python's built-in names (`len`, `print`, `range`, ...)



1. What is Scope?

Scope refers to the region of a program where a variable is accessible.

Python uses **LEGB** scope rules:

- **L** — Local: variables defined inside the current function
- **E** — Enclosing: variables in any enclosing functions
- **G** — Global: variables defined at module level
- **B** — Built-in: Python's built-in names (len, print, range, ...)

```
x = 10          # global scope
def my_func():
    y = 20      # local scope
    print(x)    # can read global x
    print(y)    # can read local y
my_func()
print(y)        # NameError: y is not defined here
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



2. Local Variables 1

Variables created **inside** a function exist only for the duration of that function call.



2. Local Variables 1

Variables created **inside** a function exist only for the duration of that function call.

```
def calculate_area(width, height):  
    area = width * height    # 'area' is local  
    return area  
  
result = calculate_area(5, 3)  
print(result)    # 15  
  
# print(area)    # NameError – 'area' doesn't exist here
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



3. Local Variables 2

Each call creates a **fresh** set of local variables:

```
def count_up(start):  
    count = start          # new 'count' each call  
    count += 1  
    return count  
  
print(count_up(0))        # 1  
print(count_up(5))        # 6
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



4. Global Variables 1

Variables defined **outside** all functions are global and can be **read** anywhere.

```
greeting = "Hello"          # global

def say_hello(name):
    print(greeting + ", " + name)  # reads global

say_hello("Alice")          # Hello, Alice
say_hello("Bob")            # Hello, Bob
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



5. Global Variables 2

Why avoid global variables?

- Functions become harder to test (hidden dependencies)
- Any part of the program can modify them accidentally
- Makes code harder to understand and debug
- Prefer passing values as **parameters** and returning **results**



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



6. The global Keyword

If you need to **assign** to a global variable inside a function, use the `global` keyword.

```
counter = 0      # global
```

```
def increment():  
    global counter      # declare we want the global  
    counter += 1        # now this modifies the global
```

```
increment()  
increment()  
increment()  
print(counter)  # 3
```



6. The global Keyword

If you need to **assign** to a global variable inside a function, use the global keyword.

```
counter = 0      # global

def increment():
    global counter    # declare we want the global
    counter += 1      # now this modifies the global

increment()
increment()
increment()
print(counter)  # 3
```

Without global, Python creates a new local variable:



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



7. The global Keyword

```
counter = 0
```

```
def broken_increment():  
    counter += 1    # UnboundLocalError!  
    # Python sees an assignment → treats as local  
    # but local 'counter' hasn't been defined yet
```

```
broken_increment()
```



7. The global Keyword

```
counter = 0
```

```
def broken_increment():  
    counter += 1    # UnboundLocalError!  
    # Python sees an assignment → treats as local  
    # but local 'counter' hasn't been defined yet
```

```
broken_increment()
```

Best practice: avoid global — pass the value in and return the updated value instead.



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



8. Scope Examples

Example 1 – Shadowing:

```
value = 100
```

```
def show():  
    value = 999    # new LOCAL variable, shadows global  
    print(value)   # 999
```

```
show()  
print(value)       # 100 – global unchanged
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



9. Scope Examples

Example 1 – Better version

```
value = 100

def show(value):
    value = 999      # update value
    print(value)     # 999
    return value

value = show(value)
print(value)         # 999 updated value
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



10. Scope Examples

Example 2 — Correct pattern (no globals needed):

```
def add_tax(price, rate=0.23):  
    tax = price * rate  
    total = price + tax  
    return total  
  
item_price = 50.00  
final_price = add_tax(item_price)  
print(f"Total: €{final_price:.2f}")    # Total: €61.50
```



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



11. Scope Best Practices

Do This	Reason
Pass data as parameters	Explicit, testable, no hidden dependencies
Return results from functions	Caller controls what to do with the result
Keep functions short and focused	Easier to reason about scope
Avoid <code>global</code> keyword	Global state leads to bugs
Use constants in <code>UPPER_CASE</code>	Signals “don’t modify this”



Outline

1. What is Scope?

2. Local Variables 1

3. Local Variables 2

4. Global Variables 1

5. Global Variables 2

6. The global Keyword

7. The global Keyword

8. Scope Examples

9. Scope Examples

10. Scope Examples

11. Scope Best Practices

12. Scope Best Practices



12. Scope Best Practices

```
# Good pattern
MAX_SCORE = 100      # module-level constant (acceptable)

def calculate_grade(score):
    percentage = (score / MAX_SCORE) * 100
    if percentage >= 70:
        return "Pass"
    return "Fail"
```



Thanks for Watching - Any questions?

